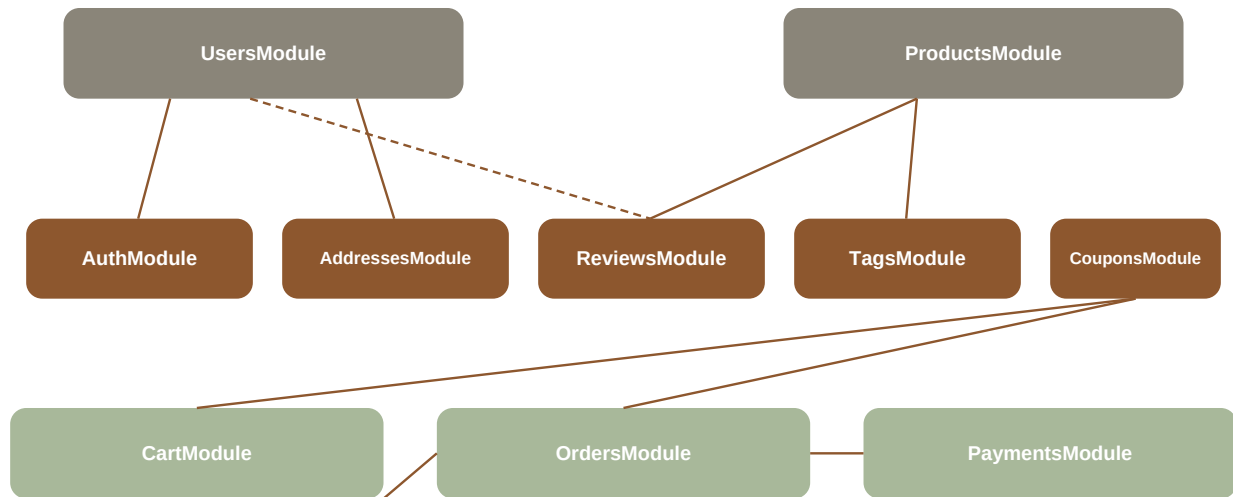


Antojarte

Diseño de la API — Nest.js

Módulos, endpoints y decisiones transversales

1. Arquitectura de módulos



UsersModule y ProductsModule son la base. AuthModule y AddressesModule dependen de UsersModule; ReviewsModule depende de ambos; TagsModule depende de ProductsModule. CouponsModule es independiente y lo consumen CartModule y OrdersModule al validar/aplicar descuentos. ProductsModule también es usado directamente por CartModule y OrdersModule a través de PRODUCT_VARIANTS (no dibujado para no sobrecargar el diagrama).

2. AuthModule

Método	Ruta	Auth	Descripción
POST	/auth/register	■■	Crea usuario, devuelve JWT + datos básicos
POST	/auth/login	■■	Valida credenciales, devuelve JWT
GET	/auth/me	■■	Usuario autenticado (para hidratar el front al cargar)

Sin refresh tokens ni logout server-side por ahora: un JWT con expiración razonable (7 días) y borrado del token en el cliente alcanza para un proyecto de práctica.

3. UsersModule

Método	Ruta	Auth	Descripción
GET	/users/me	■■	Perfil del usuario logueado
PATCH	/users/me	■■	Editar nombre, teléfono

4. AddressesModule

Método	Ruta	Auth	Descripción
GET	/addresses	■■	Direcciones guardadas del usuario
POST	/addresses	■■	Crear dirección
PATCH	/addresses/:id	■■	Editar dirección
DELETE	/addresses/:id	■■	Borrar dirección
PATCH	/addresses/:id/default	■■	Marcar como predeterminada

5. ProductsModule + TagsModule

Método	Ruta	Auth	Descripción
GET	/products	■ ■	Listado con filtros: ?tags=..&sort;=..&page;=&limit;=
GET	/products/:slug	■ ■	Detalle: variantes (x1/x6/x12), imágenes, tags, rating
GET	/tags	■ ■	Lista de tags para el sidebar de filtros

Los endpoints de administración (POST/PATCH/DELETE /products) quedan fuera de alcance por ahora — cuando se encare el panel admin, son los mismos recursos con un guard @Roles('admin').

6. ReviewsModule

Método	Ruta	Auth	Descripción
GET	/products/:slug/reviews	■ ■	Paginado, para la sección siempre visible en la ficha
POST	/products/:slug/reviews	■ ■	Crear review { rating, comment }

Cuando Orders esté funcionando, conviene validar en el service que el usuario haya comprado el producto antes de dejarlo reseñar. No bloqueante para el MVP.

7. CartModule

El módulo con la lógica más particular, por el guest checkout.

Método	Ruta	Auth	Descripción
GET	/cart	■ ■ / ■ ■	Carrito actual (lo crea si no existe)
POST	/cart/items	■ ■ / ■ ■	Agregar { variant_id, quantity }
PATCH	/cart/items/:id	■ ■ / ■ ■	Cambiar cantidad
DELETE	/cart/items/:id	■ ■ / ■ ■	Quitar item
POST	/cart/coupon	■ ■ / ■ ■	Aplicar { code } (valida monto mínimo, vigencia, límite)
DELETE	/cart/coupon	■ ■ / ■ ■	Quitar cupón aplicado
POST	/cart/merge	■ ■	Fusiona el carrito de invitado con el del usuario al hacer login

Cómo identificar el carrito de un invitado

- Al primer GET /cart sin JWT, el backend genera un guest_token (UUID) que el frontend guarda (cookie o localStorage) y reenvía en un header propio, ej. X-Guest-Token.
- Un OptionalJwtAuthGuard (custom) decodifica el JWT si viene, pero no rechaza la request si no viene — así un mismo controller sirve para ambos casos, y el service resuelve WHERE user_id = ? OR guest_token = ?.
- /cart/merge se dispara desde el frontend justo después de un login/registro exitoso, si había un guest_token activo con items.

8. CouponsModule

No expone endpoints públicos propios: la validación vive dentro de CartModule (POST /cart/coupon). Este módulo solo expone el service internamente (CouponsService.validate(code, subtotal)) para que CartModule y OrdersModule lo reutilicen. Los endpoints de administración de cupones quedan para el panel admin.

9. OrdersModule

Método	Ruta	Auth	Descripción
POST	/orders	■■/■■	Checkout: convierte el carrito actual en orden
GET	/orders	■■	Historial de pedidos del usuario logueado
GET	/orders/:id	■■	Detalle de un pedido propio
GET	/orders/guest/:id	■■	Lookup de pedido de invitado, validando ?email=coincidente

Body de POST /orders (ejemplo)

```
{
  "shipping_full_name": "...",
  "shipping_phone": "...",
  "shipping_street": "...",
  "shipping_city": "...",
  "shipping_postal_code": "...",
  "guest_email": "..." // solo si no hay JWT
}
```

Recalcular todo en el backend (crítico)

El service de POST /orders nunca debe confiar en precios/totales que mande el frontend. En una transacción de MySQL: (1) relee unit_price_snapshot de cada CART_ITEM y valida stock en PRODUCT_VARIANTS, (2) recalcula subtotal, revalida el cupón si hay uno aplicado y calcula discount_amount, (3) suma el shipping_amount fijo, (4) crea la ORDER + ORDER_ITEMS copiando los snapshots, (5) descuenta stock, (6) vacía el carrito. Todo en una única transacción para que no quede un pedido a medias si algo falla (por ejemplo, se quedó sin stock justo en ese momento).

10. PaymentsModule

Método	Ruta	Auth	Descripción
POST	/orders/:id/payment	■■/■■	Confirma el pago del pedido (simulado o gateway en modo test)
GET	/orders/:id/payment	■■/■■	Estado del pago

Para esta etapa, lo más simple es simular el pago (status: 'approved' automático o con un delay falso) en vez de integrar un gateway real, para no depender de credenciales de Stripe/MercadoPago. La tabla PAYMENTS ya está pensada para soportar una integración real sin cambios de modelo.

11. Envío (sin módulo propio)

Como el envío es un método único y fijo, es simplemente una constante (SHIPPING_FLAT_RATE) que usa OrdersModule al calcular el total. Si más adelante se suma más de un método, ahí sí se justifica un módulo propio.

12. Decisiones transversales

Validación

- DTOs + class-validator en cada endpoint de escritura (CreateAddressDto, AddCartItemDto, CreateOrderDto, etc.) con un ValidationPipe global en main.ts.

Guards

- JwtAuthGuard (estricto) para /users, /addresses, /orders (rutas de usuario logueado).
- OptionalJwtAuthGuard (custom) para /cart y POST /orders, donde tanto invitados como usuarios logueados deben poder operar.

Estructura de carpetas

- Un folder por módulo (src/auth, src/products, src/cart, etc.), cada uno con *.controller.ts, *.service.ts, *.module.ts, dto/, entities/.